

◆ CS EDUCATION ◆

🔍

프로그래밍 패러다임

25.11.07

박현정

교육 목표

- 프로그래밍 패러다임이 무엇일까?
- 선언형 패러다임
 - 함수형 프로그래밍
- 명령형 패러다임
 - 절차지향 프로그래밍
 - 객체지향 프로그래밍

패러다임이란?

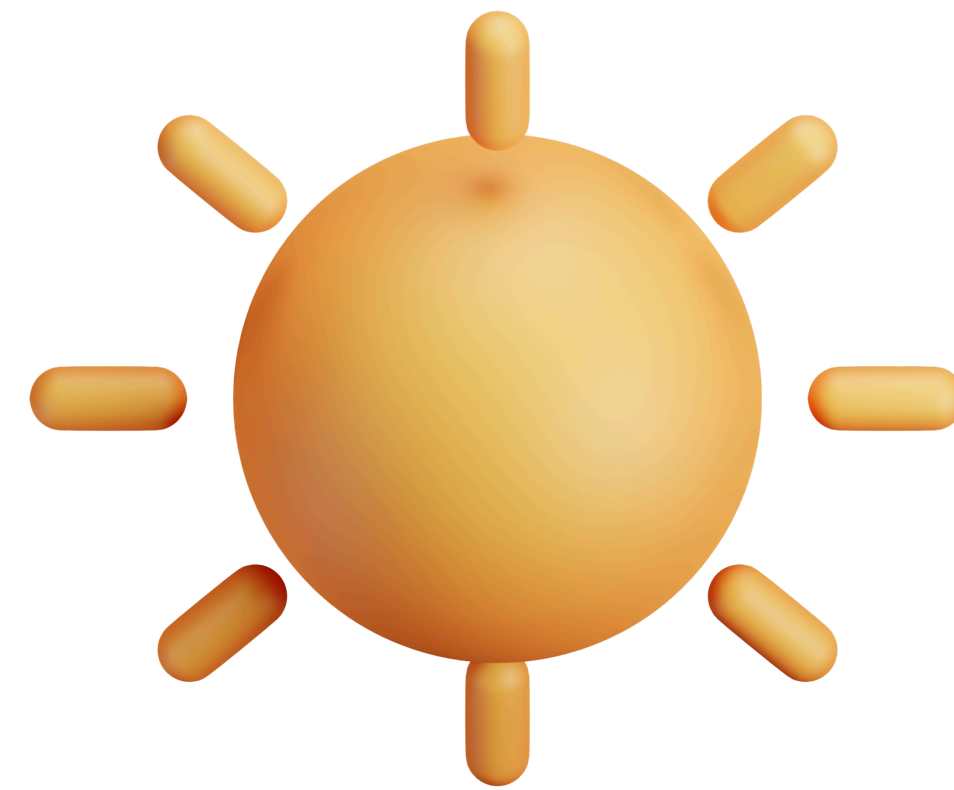
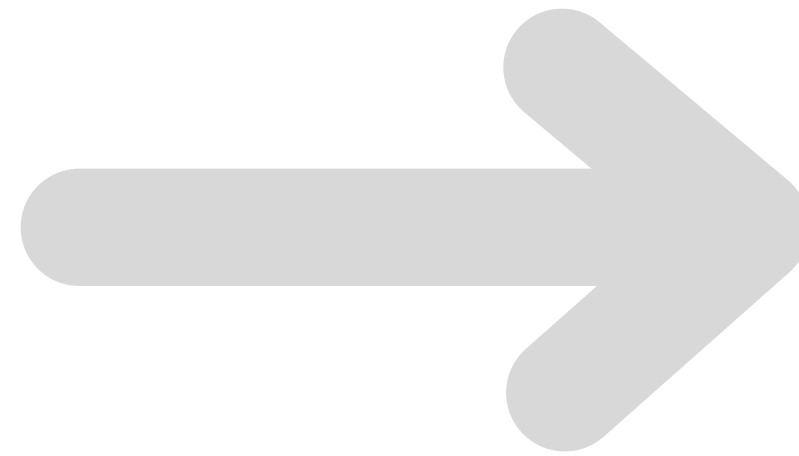
“ 어떤 한 시대 사람들의 견해나 사고를 근본적으로 규정하고 있는
테두리로서의 인식의 체계,
또는 사물에 대한 이론적인 틀이나 체계 ”

=> 세상을 바라보는 사고방식의 틀

패러다임이란?



천동설



지동설

세상을 보는 관점이 바뀌면 이해 방식도 달라진다.

패러다임이란?

패러다임 → 프로그래밍 분야

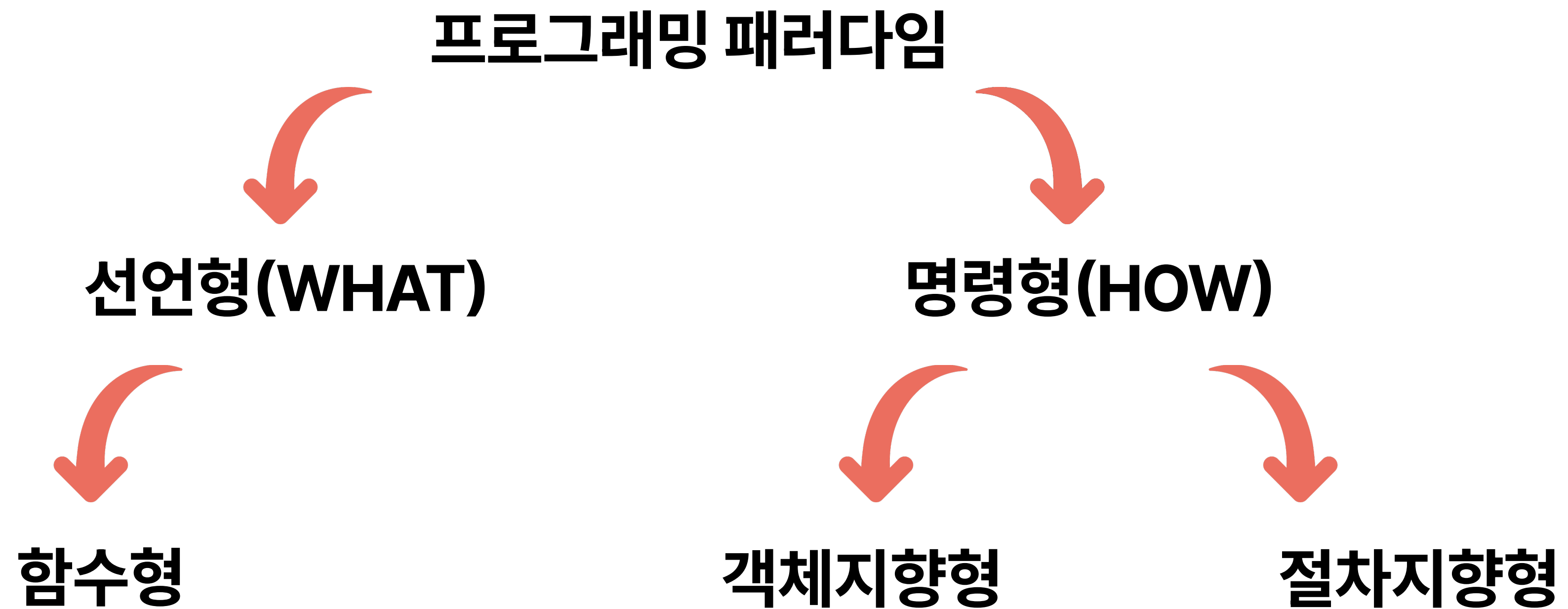
어떻게 프로그래밍할 것인지에 대한 인식의 체계를 제공해줄 수 있다.

프로그래밍 패러다임이란?

프로그램을 설계하고 구현하는 사고방식과 철학

개발의 일관된 원칙과 규칙을 제공

프로그래밍 패러다임이란?



선언형 패러다임

- **무엇(What)**을 할지에 집중
- **방법(How)**은 컴퓨터나 함수에게 맡긴다

선언형 프로그래밍

명령형 vs 선언형

```
let numbers = [1, 2, 3, 4, 5];  
let result = [];
```

```
for (let i = 0; i < numbers.length; i++) {  
  result.push(numbers[i] * 2);  
}
```

“i를 0부터 시작해 증가해가면서, 배열의 끝까지 반복하면서 2를 곱해라”

- 반복문 for문으로 직접 반복
- **어떻게 (How)**를 일일이 설명

선언형 프로그래밍

명령형 vs 선언형

```
let numbers = [1, 2, 3, 4, 5];  
let result = numbers.map(x => x * 2);
```

map 함수에게

“이 numbers 배열의 모든 원소에 2를 곱한 새로운 배열을 만들어줘” 부탁

- 무엇을(What) 할 것인지만 말함
 - 의도만 표현, 방법은 맡김

함수형 프로그래밍

순수 함수와 고차 함수를 활용하여
선언적 스타일의 프로그램을 구현하는 패러다임

“무엇을 할지”에 집중하려면, 함수가 밖에 있는 값이나 상태에 영향을 받지 않아야 한다.
즉, 외부 상태에 의존하지 않아야 한다.
외부 상태가 바뀌면 그 함수가 “무엇을 할지”가 매번 달라져서
예측하기가 어려워지기 때문이다.

함수형 프로그래밍: 순수함수란?

💡 순수함수

“같은 입력을 주면 항상 같은 결과를 내고,
함수 밖의 상태를 바꾸지 않는 함수”

```
const pure = (a, b) => {  
  return a + b;  
};
```

- 같은 입력 (2, 3)을 받으면 언제나 결과가 5로 항상 예측 가능
- 오로지 들어오는 매개변수 a, b에만 영향을 받음

함수형 프로그래밍: 순수함수란?

그렇다면 **비순수함수**는?

```
let total = 0;  
const impure = (a, b) => {  
  total += a + b;  
  return total;  
};
```

- impure 함수 외부에 선언되어있는 변수 total
- impure 함수 내에서이 외부의 total의 값을 바꿔버리고 있다.
- 따라서 같은 입력 (2, 3)을 받더라도 total의 값이 달라짐

→ 함수가 외부 상태에 의존하거나 영향을 미치는 것(side effect),
즉 **부수효과**를 없애는 게 함수형의 핵심

함수형 프로그래밍: 고차함수란?

💡 고차함수

“함수를 일반 데이터나 값처럼
인자로 받거나 반환할 수 있는 함수”

💡 일급 객체

고차함수를 사용하기 위해서는
함수가 ‘일급 객체’로 취급되어야 한다.

함수형 프로그래밍: 일급 객체(1)

(1) 변수나 메서드에 값을 할당하듯이 함수를 할당할 수 있다.

```
// 함수도 값처럼 변수에 저장  
const sayHello = function() {  
  console.log("Hello!");  
}
```

```
// 변수 이름으로 호출 가능  
sayHello(); // Hello!
```

→ sayHello라는 변수에 함수를 할당하고,
변수 이름으로 함수를 호출한다.

함수형 프로그래밍: 일급 객체(2)

(2) 함수 안에 함수를 매개변수로 담을 수 있다.

```
function greet(func) { // 함수(func)를 매개변수로 받음
  console.log("인사 시작");
  func(); // 전달받은 함수 실행
  console.log("인사 끝");
}

function sayHi() {
  console.log("안녕!");
}

// 함수 자체를 인자로 전달
greet(sayHi);
```

greet(sayHi);
→ greet 함수는 sayHi 함수를 매개변수로 받고 있고,
내부 로직에서 전달받은 이 sayHi 함수를 사용하는 전형적인 고차함수이다.

함수형 프로그래밍: 일급 객체(3)

(3) 함수가 함수를 반환할 수 있다.

```
function makeAdder(x) {  
  // 내부에서 새 함수를 만들어 반환  
  return function(y) {  
    return x + y;  
  }  
}
```

→ makeAdder 함수는 내부에 새 함수를 만들어 반환하는 고차함수

```
const add5 = makeAdder(5);  
console.log(add5(3));  
console.log(add5(10));
```

// x가 5로 고정된 함수 생성
// 8
// 15

→ makeAdder(5) 호출 시
인자로 받은 x값(=5)를 기억하고 있는
내부 함수가 add5에 반환

함수형 프로그래밍: 요약

항상 같은 입력에 같은 결과를 내는 작은 **순수함수**들을
레고 블록처럼 조립해서 로직을 만든다.
고차 함수는 이런 블록들을 더 유연하게 다루게 해주는 도구

명령형 프로그래밍

- 어떻게(How) 문제를 해결할지 직접 지시
- 컴퓨터에게 “무엇을 먼저 하고, 그 다음엔 뭘 할지”를 순서대로 명령

명령형 프로그래밍

```
let numbers = [1, 2, 3, 4, 5];  
let result = [];  
  
for (let i = 0; i < numbers.length; i++) {  
  result.push(numbers[i] * 2);  
}
```

- 프로그램의 흐름, 즉 제어구조나 반복, 조건 등을 직접 명시
- 변수의 값이 실행 과정 중에 계속 변함
- 사람이 이 코드가 “어떻게 동작하는지” 단계별로 명확히 알 수 있음

절차지향 & 객체지향 패러다임

절차지향 프로그래밍

💡 절차지향 프로그래밍

“순서대로 처리하는 절차 중심의 사고방식”

→ 하나의 프로그램을 여러 개의 작업 단계(절차)로 나누고,
각 단계에서 무슨 일을 할지 구체적으로 지시

절차지향 프로그래밍

```
function boilWater() {  
  console.log("물을 끓인다");  
}  
function addNoodles() {  
  console.log("면을 넣는다");  
}  
function addSoup() {  
  console.log("스프를 넣는다");  
}  
function eat() {  
  console.log("라면을 먹는다");  
}
```

```
boilWater();  
addNoodles();  
addSoup();  
eat();
```



위에서 아래로 차례차례 실행되는 구조

라면 끓이기 시뮬레이션을 절차형 코드로 짤다면?

절차지향 프로그래밍

특징

- 프로그램이 “무엇을 할까”보다 “어떤 순서로 할까”에 초점을 둔다.
- 함수(절차) 단위로 나뉘서 코드의 재사용이 가능하다.
- 코드의 흐름이 명확하지만, 프로그램이 커질 수록 데이터와 함수가 분리되어 유지보수가 어려워진다.

객체지향 프로그래밍

💡 객체지향 프로그래밍

“데이터(속성)과 그 데이터를 처리하는 메서드(기능)을 하나의 객체로 묶어,
각각의 객체 하나하나가 자신의 데이터와 행동을 가지고
일을 처리하도록 설계하는 방식”

→ 절차지향이 “무엇을 먼저 할까?”에 집중했다면,
객체지향은 “누가 그 일을 할까?”에 초점을 둔다.

객체지향 프로그래밍

```
class Car {  
  startEngine() {  
    console.log("엔진 시동!");  
  }  
  pressAccelerator() {  
    console.log("부릉부릉~ 속도 증가!");  
  }  
  brake() {  
    console.log("멈췄습니다.");  
  }  
}
```

```
const myCar = new Car();  
myCar.startEngine();  
myCar.pressAccelerator();  
myCar.brake();
```

자동차 시뮬레이션을 코드로 짤다면?

Car

→ '자동차'의 설계도로서,
안에 엔진 시동을 켜는 행동, 엑셀을 밟는 행동,
브레이크를 밟는 행동을 모아두고 있다.

myCar

→ 그 설계도로 만든 실제 자동차 한 대

즉, myCar라는 객체는 스스로 엔진을 켜고, 달리고,
멈추는 것처럼 스스로 일을 하는 주체로 표현된다.

객체지향 프로그래밍

핵심 특징

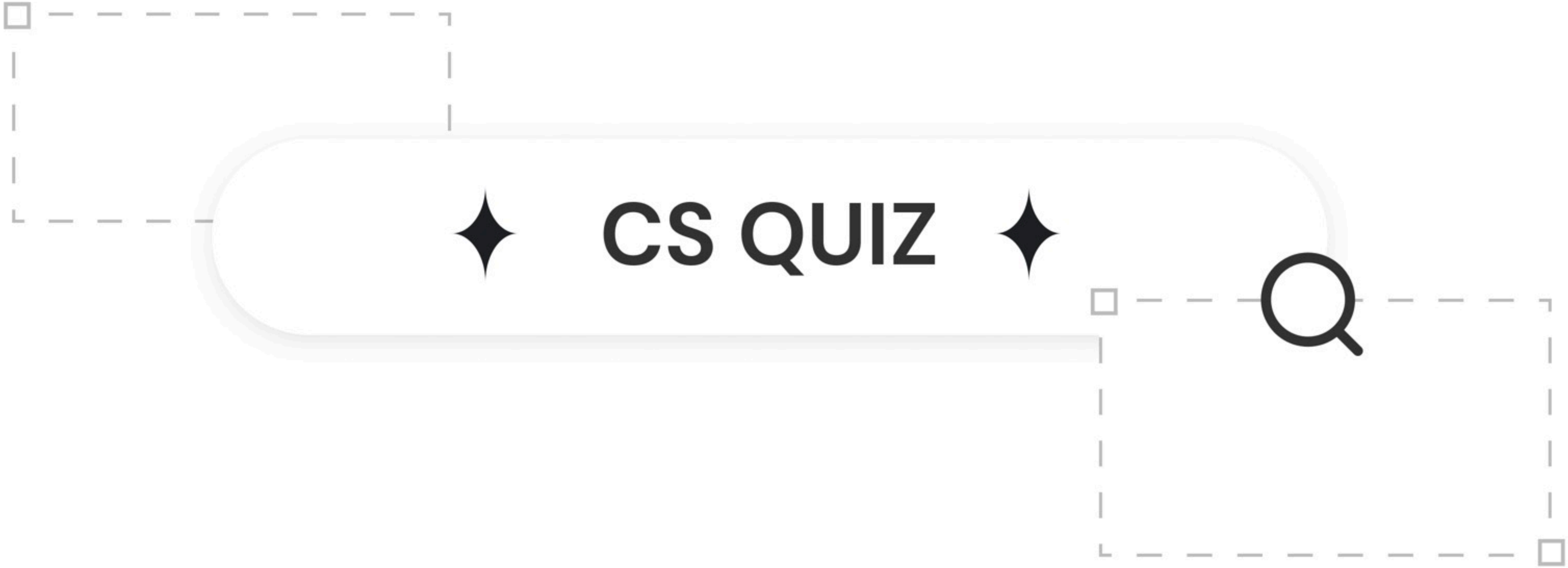
- **캡슐화:** 관련된 데이터와 기능을 한 객체 안에 묶는다.
- **상속:** 기존의 객체를 확장해 새로운 객체를 만든다.
- **다형성:** 같은 메서드라도 객체에 따라서 다르게 작동한다.
- **추상화:** 불필요한 세부 내용은 감추고 필요한 기능만 보이게 한다.

마무리

선언형 → “결과에 집중”
명령형 → “과정에 집중”
절차지향 → “순서대로 실행”
객체지향 → “객체들이 협력”

어떤 패러다임이 가장 좋을까?
→ 답은 “그런 것은 없다”

여러 패러다임을 조합해 상황과
맥락에 따라 장점만을 취해 개발



Q.1

"프로그래밍 패러다임"에 대한 설명으로 가장 올바른 것을 고르세요.

- ① 특정 언어(Java, Python 등)에만 존재하는 고유한 문법 규칙을 의미한다.
- ② 프로그램을 설계하고 구현하는 사고방식이자 철학으로, '어떻게' 풀지(명령형)와 '무엇을' 풀지(선언형)의 관점 차이를 포함한다.
- ③ 선언형 패러다임은 절차지향과 객체지향으로 나뉜다.
- ④ 객체지향 프로그래밍은 부수 효과가 없는 '순수 함수'의 조합을 가장 중요하게 생각한다.



A.1

"프로그래밍 패러다임"에 대한 설명으로 가장 올바른 것을 고르세요.

- ① 특정 언어(Java, Python 등)에만 존재하는 고유한 문법 규칙을 의미한다.
- ② 프로그램을 설계하고 구현하는 사고방식이자 철학으로, '어떻게' 풀지(명령형)와 '무엇을' 풀지(선언형)의 관점 차이를 포함한다.
- ③ 선언형 패러다임은 절차지향과 객체지향으로 나뉜다.
- ④ 객체지향 프로그래밍은 부수 효과가 없는 '순수 함수'의 조합을 가장 중요하게 생각한다.

정답: 2번

Q.2

"라면 끓이기"라는 작업을 프로그래밍한다고 할 때, 두 개발자의 사고방식이
다릅니다. 각 방식과 가장 적합한 패러다임을 짚으세요.

희정 : "물을 끓인다" → "면을 넣는다" → "스프를 넣는다"처럼, 작업의 순서를 중심
으로 프로그램을 구성해야겠어.

현정 : "누가 그 일을 하지?"를 고민. "자동차"가 "시동을 켜다", "엑셀을 밟는다"처럼,
주체가 스스로 자신의 일을 하도록 설계해야겠어.

- ① 희정: 객체지향 / 현정: 절차지향
- ③ 희정: 절차지향 / 현정: 객체지향

- ② 희정: 함수형 / 현정: 객체지향
- ④ 희정: 선언형 / 현정: 명령형



A.2

"라면 끓이기"라는 작업을 프로그래밍한다고 할 때, 두 개발자의 사고방식이
다릅니다. 각 방식과 가장 적합한 패러다임을 짚으세요.

희정 : "물을 끓인다" → "면을 넣는다" → "스프를 넣는다"처럼, 작업의 순서를 중심
으로 프로그램을 구성해야겠어.

현정 : "누가 그 일을 하지?"를 고민. "자동차"가 "시동을 켜다", "엑셀을 밟는다"처럼,
주체가 스스로 자신의 일을 하도록 설계해야겠어.

① 희정: 객체지향 / 현정: 절차지향

③ 희정: 절차지향 / 현정: 객체지향

② 희정: 함수형 / 현정: 객체지향

④ 희정: 선언형 / 현정: 명령형

정답: 3번

Q.3

다음 중 선언형과 명령형 패러다임의 구분 기준으로 가장 핵심적인 차이를 고르세요.

- ① 코드의 길이
- ② '무엇을' vs '어떻게'에 대한 관점
- ③ 함수 사용 여부
- ④ 컴파일 방식



A.3

다음 중 선언형과 명령형 패러다임의 구분 기준으로 가장 핵심적인 차이를 고르세요.

- ① 코드의 길이
- ② '무엇을' vs '어떻게'에 대한 관점
- ③ 함수 사용 여부
- ④ 컴파일 방식

정답: 2번

Q.4

절차지향 프로그래밍의 특징으로 옳지 않은 것을 고르세요.

- ① 프로그램이 "어떤 순서로 할까"에 초점을 둔다
- ② 함수(절차) 단위로 나눠서 코드의 재사용이 가능하다
- ③ 프로그램이 커질수록 유지보수가 쉬워진다
- ④ 코드의 흐름이 명확하다



A.4

절차지향 프로그래밍의 특징으로 옳지 않은 것을 고르세요.

- ① 프로그램이 "어떤 순서로 할까"에 초점을 둔다
- ② 함수(절차) 단위로 나눠서 코드의 재사용이 가능하다
- ③ 프로그램이 커질수록 유지보수가 쉬워진다
- ④ 코드의 흐름이 명확하다

정답: 3번

Q.5

객체지향 프로그래밍의 특징으로 옳지 않은 것을 고르세요.

- ① 추상화
- ② 상속
- ③ 다형성
- ④ 정규화



A.5

객체지향 프로그래밍의 특징으로 옳지 않은 것을 고르세요.

- ① 추상화
- ② 상속
- ③ 다형성
- ④ 정규화

정답: 4번

Q.6

아래 중 선언형과 관련이 없는 것은? (정답 2개)

a) SQL - SELECT name FROM users WHERE age > 20;

b) java 클래스의 상속 구조

c) 함수가 외부 변수를 사용하지 않는다

d) JavaScript의 map()을 이용한 배열 변환

e) 예시 코드



```
let total = 0;  
const cotato = (a, b) => {  
  total += a + b;  
  return total;  
};
```


A.6

아래 중 선언형과 관련이 없는 것은? (정답 2개)

a) SQL - SELECT name FROM users WHERE age > 20;

b) java 클래스의 상속 구조

c) 함수가 외부 변수를 사용하지 않는다

d) JavaScript의 map()을 이용한 배열 변환

e) 예시 코드



```
let total = 0;  
const cotato = (a, b) => {  
  total += a + b;  
  return total;  
};
```

정답: b, e

Q.7

[주관식]

당신은 Java 언어를 사용하여 캐릭터가 공격, 방어, 레벨업 기능을 수행하고 몬스터나 아이템과 상호작용하는 게임 시스템을 구현해야 한다.

이때, 어떤 프로그래밍 패러다임을 사용하는 것이 더 적절한가?



A.7

[주관식]

당신은 Java 언어를 사용하여 캐릭터가 공격, 방어, 레벨업 기능을 수행하고 몬스터나 아이템과 상호작용하는 게임 시스템을 구현해야 한다.

이때, 어떤 프로그래밍 패러다임을 사용하는 것이 더 적절한가?

정답: 객체지향 프로그래밍

Q.8

다음 중 함수형 프로그래밍의 핵심 원칙이 아닌 것을 고르세요.

- ① 순수 함수로 외부 상태를 변경하지 않는다.
- ② 같은 입력에는 항상 같은 출력을 낸다.
- ③ 부수 효과(side effect)를 적극적으로 활용한다.
- ④ 함수를 값처럼 다루는 고차함수를 사용할 수 있다.



A.8

다음 중 함수형 프로그래밍의 핵심 원칙이 아닌 것을 고르세요.

- ① 순수 함수로 외부 상태를 변경하지 않는다.
- ② 같은 입력에는 항상 같은 출력을 낸다.
- ③ 부수 효과(side effect)를 적극적으로 활용한다.
- ④ 함수를 값처럼 다루는 고차함수를 사용할 수 있다.

정답: 3번

Q.9

다음 중 Javascript에서 함수가 일급 객체로서 가지는 특징으로
알맞지 않은 것을 고르세요.

- ① 함수를 변수나 상수에 할당할 수 있다.
- ② 함수를 다른 함수의 인자로 전달할 수 있다.
- ③ 함수를 다른 함수의 반환값으로 사용할 수 있다.
- ④ 함수는 프로퍼티를 가질 수 없다.



A.9

다음 중 Javascript에서 함수가 일급 객체로서 가지는 특징으로
알맞지 않은 것을 고르세요.

- ① 함수를 변수나 상수에 할당할 수 있다.
- ② 함수를 다른 함수의 인자로 전달할 수 있다.
- ③ 함수를 다른 함수의 반환값으로 사용할 수 있다.
- ④ 함수는 프로퍼티를 가질 수 없다.

정답: 4번

Q.10

다음 중 설명을 읽고 패러다임의 성격이 같은 것끼리 묶은 것을 고르세요.

- ㄱ. 데이터와 함수가 따로 존재하므로 데이터의 수정이 일어나면 관련 함수를 모두 수정해야 한다.
- ㄴ. 함수를 매개변수로 담거나 return 값으로 반환할 수 있어야 한다.
- ㄷ. 어떻게 문제를 해결할지 직접 지시하는 방식이다.
- ㄹ. 같은 입력을 주면 항상 같은 결과를 내고, 외부 상태에 영향받지 않는다.

① ㄱㄴ / ㄷㄹ

② ㄱㄷ / ㄴㄹ

③ ㄱㄹ / ㄴㄷ

④ ㄱㄴㄷ / ㄹ

A.10



정답: 2번

다음 중 설명을 읽고 패러다임의 성격이 같은 것끼리 묶은 것을 고르세요.

ㄱ. 데이터와 함수가 따로 존재하므로 데이터의 수정이 일어나면 관련 함수를 모두 수정해야 한다. - **절차형**

ㄴ. 함수를 매개변수로 담거나 return 값으로 반환할 수 있어야 한다.

- **선언형(일급객체)**

ㄷ. 어떻게 문제를 해결할지 직접 지시하는 방식이다. - **명령형**

ㄹ. 같은 입력을 주면 항상 같은 결과를 내고, 외부 상태에 영향받지 않는다.

- **선언형(순수함수)**

① ㄱ ㄴ / ㄷ ㄹ

② ㄱ ㄷ / ㄴ ㄹ

③ ㄱ ㄹ / ㄴ ㄷ

④ ㄱ ㄴ ㄷ / ㄹ